

Hibernate ddl-auto

A practical Spring Boot guide to validate, update, create, create-drop, and none

SAFE CHECK	DEV CONVENIENCE	DISPOSABLE DATA
validate / none	update	create / create-drop

Spring Boot property

```
spring.jpa.hibernate.ddl-auto=<value>
```

This setting controls how Hibernate treats the database schema when the application starts. The create-drop option also performs an action when Hibernate shuts down normally.

Starting point

Hibernate derives an expected database schema from entity mappings such as `@Entity`, `@Table`, `@Id`, and `@Column`.

```
@Entity
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private BigDecimal price;
}
```

The entity above implies a product table with id, name, and price columns. Each ddl-auto mode decides what Hibernate should do when the actual database differs from this expectation.

1. validate

```
spring.jpa.hibernate.ddl-auto=validate
```

What does it do?

Checks whether the existing schema matches the entity mappings. It does not create, alter, or delete database objects.

How does it work?

At startup, Hibernate checks required tables, columns, compatible column types, sequences, and related mapped objects. A detected mismatch normally stops application startup with a schema-validation exception.

Simple example

If Product contains a price field but product.price is missing in the database, Hibernate reports the mismatch. It does not add the missing column.

Real-world use case

Use it in production when Flyway, Liquibase, or a DBA applies migrations. Hibernate then acts as a startup safety check that catches missing or incompatible schema changes.

2. update

```
spring.jpa.hibernate.ddl-auto=update
```

What does it do?

Compares entity mappings with the current schema and attempts to bring the schema up to date while preserving existing tables and data where possible.

How does it work?

Hibernate reads database metadata and generates supported DDL. It can create missing tables, add missing columns, create sequences or constraints, and perform certain alterations. Exact behavior depends on the Hibernate version and database dialect.

Simple example

After adding private String description to Product, Hibernate may run: ALTER TABLE product ADD COLUMN description VARCHAR(255);

Real-world use case

Useful during local development and for small prototypes. A developer can change an entity and restart the application without manually writing a migration for every experiment.

Production warning: update is not a full migration system. Renames may be interpreted as add-new-column operations, destructive cleanup is not reliably managed, and data transformations require deliberate migrations.

What happens in the requested example?

With ddl-auto=update, Hibernate inspects the live schema, compares it with the mapped entities, and applies supported additions or alterations. It does not intentionally recreate the whole database. This is convenient for development, but production changes should normally be reviewed, versioned migrations.

3. create

```
spring.jpa.hibernate.ddl-auto=create
```

What does it do?

Drops Hibernate-managed schema objects and recreates them when the application starts. Existing data in those objects is lost.

How does it work?

For Spring's ddl-auto property, the value is interpreted through Hibernate's hbm2ddl behavior: drop the existing mapped schema, then generate fresh tables, sequences, and constraints from the entities.

Simple example

If product contains a Laptop row, startup drops and recreates the product table. The new table is empty and the Laptop row is gone.

Real-world use case

Useful for integration-test environments, classroom exercises, demonstrations, and early prototypes where every startup should begin with a clean schema.

Never point create at a production database containing valuable data. It can erase the mapped schema during startup.

Important terminology detail

Hibernate documents an ambiguity: legacy hbm2ddl 'create' drops and recreates the schema, while the JPA-standard database action named 'create' means create-only. This guide describes spring.jpa.hibernate.ddl-auto.

4. create-drop

```
spring.jpa.hibernate.ddl-auto=create-drop
```

What does it do?

Creates a fresh schema at startup and drops it again when Hibernate shuts down normally.

How does it work?

Startup performs drop then create. When the SessionFactory or EntityManagerFactory closes normally, Hibernate issues another drop operation.

Simple example

With jdbc:h2:mem:testdb, tests start with empty Hibernate-generated tables. When the test context closes, Hibernate drops those tables.

Real-world use case

Ideal for automated integration tests, temporary in-memory databases, and disposable environments that must leave no persistent schema behind.

The final drop depends on normal shutdown. A forced process termination or crash can prevent shutdown cleanup.

5. none

```
spring.jpa.hibernate.ddl-auto=none
```

What does it do?

Disables Hibernate's automatic schema management. Hibernate does not validate, create, update, or drop schema objects.

How does it work?

Hibernate builds its mappings but performs no schema-management action. A missing table might therefore be discovered only when application code later executes SQL against it.

Simple example

If product does not exist, application startup may still succeed. Calling `productRepository.findAll()` later can fail with a 'table does not exist' database error.

Real-world use case

Use it when Flyway, Liquibase, database scripts, or a DBA fully owns schema management and Hibernate should not even validate it. Choose `validate` instead when startup compatibility checking is desired.

Comparison

Value	What does it do?	Real-world use case
validate	Checks schema compatibility without making changes.	Production with controlled migrations plus a startup safety check.
update	Adds or alters supported objects while trying to preserve data.	Local development and prototypes with frequent entity changes.
create	Drops and recreates the mapped schema at startup.	Tests, demos, training, or disposable databases.
create-drop	Recreates at startup and drops again on normal shutdown.	Automated tests and temporary in-memory databases.
none	Performs no schema-management action.	Externally managed production or legacy schemas.

A practical profile setup

```
# application-dev.properties
spring.jpa.hibernate.ddl-auto=update

# application-test.properties
spring.jpa.hibernate.ddl-auto=create-drop

# application-prod.properties
spring.jpa.hibernate.ddl-auto=validate
```

Rule of thumb

Use update, create, and create-drop mainly for development or testing. In production, prefer reviewed migrations together with validate, or use none when validation is intentionally handled elsewhere.

References

Hibernate ORM Action documentation: <https://docs.hibernate.org/orm/6.3/javadocs/org/hibernate/tool/schema/Action.html>
 Hibernate ORM Introduction, schema management:
https://docs.jboss.org/hibernate/orm/7.1/introduction/pdf/Hibernate_Introduction.pdf